

Sistema de Gestion de Consultorio Medico

Documentacion Completa del Sistema

Framework: Django (Python)

Frontend: HTML5 + Tailwind CSS (CDN)

Base de datos: SQLite3 (dev) / PostgreSQL (produccion)

Ley 25.326: Consentimiento de datos personales

Ley 26.529: Historia clinica - Retencion 10 anios

Ubicacion: C:\Users\User\Desktop\IA_developer\CONSULTORIOS

1. ARQUITECTURA DEL SISTEMA

1.1 Stack Tecnológico

Componente	Tecnología	Version/Detalle
Backend	Django (Python)	>=4.2, <5.1
Frontend	HTML5 + Tailwind CSS	CDN (sin framework JS)
BD Desarrollo	SQLite3	db.sqlite3
BD Produccion	PostgreSQL	via dj-database-url
Servidor WSGI	Gunicorn	>=21.2.0
Estilos	Tailwind CSS + Font Awesome	CDN
Fuentes	Google Fonts Inter	CDN

1.2 Dependencias (requirements.txt)

Paquete	Version	Uso
Django	>=4.2, <5.1	Framework web principal
Pillow	>=10.0.0	Manejo de imagenes
django-otp	>=1.3.0	Autenticacion OTP (preparado)
qrcode	>=7.4.2	Generacion de QR codes
reportlab	>=4.0.0	Generacion de PDFs
django-cors-headers	>=4.3.0	Headers CORS
gunicorn	>=21.2.0	Servidor WSGI produccion
whitenoise	>=6.5.0	Archivos estaticos
dj-database-url	>=2.1.0	URL BD variable entorno

1.3 Estructura de Carpetas

CONSULTORIOS/

manage.py	- CLI de Django
requirements.txt	- Dependencias Python
Procfile	- Despliegue Heroku/Render
db.sqlite3	- Base de datos SQLite (dev)
generar_pdf_propuesta.py	- Generador PDF comercial
config/	- Configuración del proyecto Django
settings.py	- Settings principales
urls.py	- URL router principal
wsgi.py / asgi.py	- Puntos de entrada WSGI/ASGI
apps/	- Aplicaciones de dominio
configuracion/	- Configuración Sistema (singleton)

usuarios/	- Sedes, Consultorios, Medicos, Especialidades
pacientes/	- Pacientes y Obras Sociales
turnos/	- Turnos y Facturacion
historias_clinicas/	- Historia clinica (soft-delete)
auditoria/	- Log de auditoria
core/	- Vistas principales + servicios
templates/	- Templates HTML5
base.html	- Layout base
registration/login.html	- Login
recepcion/dashboard.html	- Panel recepcion
medicos/agenda.html	- Agenda medica
pacientes/	- Historia clinica
turnero/pantalla.html	- Turnero Smart TV
static/	- Archivos estaticos (CDN)
DOCS/	- Documentacion del proyecto

2. MODELO DE DATOS (12 modelos)

2.1 ConfiguracionSistema (app: configuracion)

Campo	Tipo	Detalle
nombre_consultorio	CharField(200)	Default: "Consultorio Medico San Lucas"
max_medicos_permitidos	IntegerField	Default: 2 (cupo licencia)
telefono_whatsapp_empresa	CharField(50)	Numero WhatsApp empresa
direccion_consultorio	CharField(300)	Direccion fisica
modo_demo	BooleanField	True = modo demostracion
fecha_actualizacion	DateTimeField	auto_now

Método get_solo() - Patron Singleton: crea o retorna la única instancia (id=1).

2.2 Sede (app: usuarios)

Campo	Tipo	Detalle
nombre	CharField(150)	Nombre de la sede
direccion	CharField(250)	Direccion fisica
telefono	CharField(50)	Telefono de contacto
activa	BooleanField	Si la sede esta operativa

2.3 Consultorio (app: usuarios)

Campo	Tipo	Detalle
nombre_numero	CharField(100)	Ej: "Consultorio 102 - Planta Alta"
sede	FK -> Sede	Relacion 1 a M con Sede

2.4 Especialidad (app: usuarios)

Campo	Tipo	Detalle
nombre	CharField(150)	Unico. Ej: Cardiologia, Pediatria
descripcion	TextField	Descripcion opcional

2.5 Medico (app: usuarios)

Campo	Tipo	Detalle
user	OneToOne -> User	Vinculado 1:1 a User de Django
nombre_completo	CharField(200)	Nombre completo del medico
matricula_profesional	CharField(50)	Matricula Nacional o Provincial

Sistema de Gestión de Consultorio Médico - Documentación Completa

especialidades	MME -> Especialidad	Relacion M a M
telefono_whatsapp	CharField(50)	Con codigo de pais (+549...)
activo	BooleanField	Si esta activo en el sistema
google_calendar_id	CharField(255)	Preparado para sync futura

Validacion clean(): respeta el cupo max_medicos_permitidos de ConfiguracionSistema. Si se alcanza el limite, lanza ValidationError.

2.6 MedicoSede (app: usuarios)

Campo	Tipo	Detalle
medico	FK -> Medico	Medico que atiende
sede	FK -> Sede	Sede donde atiende
dias_atencion	CharField(100)	"Lunes, Miercoles, Viernes"
horario_inicio	TimeField	Hora de inicio
horario_fin	TimeField	Hora de fin
duracion_turno_default	IntegerField	Default: 30 min

2.7 Paciente (app: pacientes)

Campo	Tipo	Detalle
nombre_completo	CharField(200)	Nombre completo
dni	CharField(20)	Unico - Documento de Identidad
telefono	CharField(50)	Movil para WhatsApp
email	EmailField	Email opcional
fecha_nacimiento	DateField	Fecha de nacimiento
obra_social	FK -> ObraSocial	Obra social/prepaga
numero_afiliado	CharField(100)	Credencial de afiliado
datos_contacto_adicionales	TextField	Contacto de emergencia
consentimiento_datos	BooleanField	Ley 25.326 (default True)
consentimiento_fecha	DateTimeField	auto_now_add
consentimiento_ip	GenericIPAddressField	IP de consentimiento
proxima_fecha_recall	DateField	Proximo control preventivo
fecha_registro	DateTimeField	auto_now_add

2.8 ObraSocial (app: pacientes)

Campo	Tipo	Detalle
nombre	CharField(200)	Unico - Nombre de la obra social
codigo_nomenclador	CharField(50)	Codigo nomenclador nacional

2.9 Turno (app: turnos)

Campo	Tipo	Detalle
medico	FK -> Medico	Medico asignado

Sistema de Gestión de Consultorio Médico - Documentación Completa

paciente	FK -> Paciente	Paciente que solicita
sede	FK -> Sede	Sede del turno
consultorio	FK -> Consultorio	Sala fisica (opcional)
fecha	DateField	Fecha del turno
hora	TimeField	Hora del turno
duracion_minutos	IntegerField	Default: 30 min
estado	CharField(20)	Estados: PENDIENTE, CONFIRMADO, CANCELADO, ATENDIDO, AUSENTE, EN_ESPERA
notificado_wsp_medico	BooleanField	Notificacion WhatsApp enviada
notificado_wsp_paciente	BooleanField	Notificacion WhatsApp enviada
en_lista_espera	BooleanField	En lista de espera
creado_por	FK -> User	Usuario que creo el turno
notas	TextField	Notas adicionales
fecha_creacion	DateTimeField	auto_now_add

2.10 FacturaConsulta (app: turnos)

Campo	Tipo	Detalle
turno	OneToOne -> Turno	Factura vinculada al turno
tipo_comprobante	CharField(20)	Factura A, B, C (AFIP)
cae	CharField(50)	Codigo Autorizacion Electronica AFIP
monto	DecimalField(12,2)	Monto de la consulta
fecha_emision	DateTimeField	auto_now_add

2.11 HistoriaClinica (app: historias_clinicas)

Campo	Tipo	Detalle
paciente	FK -> Paciente	Paciente de la consulta
medico	FK -> Medico	Medico que atendio (RESTRICT)
fecha_consulta	DateTimeField	Fecha y hora de la consulta
motivo_consulta	TextField	Motivo reportado por paciente
notas_evolucion	TextField	Examen fisico y observaciones
diagnostico	TextField	Diagnostico o hipotesis
tratamiento_prescrito	TextField	Indicaciones medicas
receta_electronica	TextField	Preparado para receta digital
is_deleted	BooleanField	Soft-delete (Ley 26.529)
deleted_at	DateTimeField	Fecha de borrado logico

Soft-delete: El manager por defecto (objects) solo muestra registros activos. all_with_deleted muestra todos. delete() marca is_deleted=True en vez de borrar fisicamente. Cumple Ley 26.529: retencion minima 10 anios.

2.12 LogAuditoria (app: auditoria)

Campo	Tipo	Detalle
usuario	FK -> User	Usuario que realizo la accion
accion	CharField(50)	CREAR / EDITAR / ELIMINAR / VER / EXPORTAR
entidad	CharField(100)	Ej: HistoriaClinica, Paciente, Turno
entidad_id	CharField(50)	ID del registro afectado
detalles	TextField	Descripcion de la accion
timestamp	DateTimeField	auto_now_add
ip_origen	GenericIPAddressField	IP del usuario

3. VISTAS Y URLS

3.1 Rutas Principales

URL	Vista	Funcionalidad	Auth
/	home_redirect	Redirige segun perfil del usuario	No
/admin/	Django Admin	Panel de administracion	Staff
/accounts/login/	LoginView	Formulario de login	No
/accounts/logout/	LogoutView	Cerrar sesion	No
/recepcion/	recepcion_dashboard	Dashboard recepcion con KPIs	Si
/medico/agenda/	medico_agenda	Agenda mobile del medico	Si
/turnos/<id>/cambiar-estado/	cambiar_estado_turno	POST cambiar estado	Si
/pacientes/<id>/historia-clinica/	paciente_historia_clinica	Ver/crear HC	Si
/turnero/	turnero_pantalla	Pantalla Smart TV sala espera	No

3.2 Vista: recepcion_dashboard

Dashboard multi-medico para recepcion. Filtra turnos por fecha y medico. Muestra KPIs (total, en espera, confirmados, atendidos, cancelados), tabla de turnos con acciones rapidas ("Anunciar Llegada", "HC").

3.3 Vista: medico_agenda

Agenda mobile-first del medico. Muestra cards de turnos del dia con botones "Iniciar/Atendido" e "Historia Clinica". Si el usuario no es medico pero es staff, accede al agenda del primer medico activo.

3.4 Vista: turnero_pantalla

Pantalla publica para Smart TV en sala de espera. Auto-refresh cada 10 segundos. Muestra reloj en vivo, pacientes llamados (verde), lista de espera. No requiere autenticacion.

3.5 Vista: paciente_historia_clinica

Muestra historial clinico del paciente + formulario para nueva evolucion medica. Registra auditoria de acceso (Ley 26.529).

3.6 Vista: cambiar_estado_turno

Endpoint POST para cambiar estado de un turno. Registra cambio en LogAuditoria con usuario, accion, entidad y IP.

4. FUNCIONALIDADES IMPLEMENTADAS

> Panel de Recepcion

Dashboard completo con filtros por fecha/medico, estadísticas en tiempo real y acciones rapidas (anunciar llegada, ver HC).

> Agenda Mobile del Medico

Vista mobile-first con tarjetas de turnos, 1 toque para cambiar estado. Iniciar, Atendido, Historia Clinica.

> Turnero TV Sala de Espera

Pantalla publica para Smart TV con reloj en vivo, pacientes llamados en verde, auto-refresh cada 10 segundos.

> Historia Clinica Legal

Formulario de evolucion medica + timeline historica + soft-delete obligatorio (Ley 26.529, retencion 10 anios).

> Validacion de Superposicion

El modelo Turno valida que no haya dos turnos simultaneos para el mismo medico en la misma fecha/hora.

> Regla de Cupo/Licencia

ConfiguracionSistema limita la cantidad de medicos activos (default: 2). Validacion en clean() del modelo Medico.

> Auditoria Completa

LogAuditoria registra cada cambio de estado de turno y cada acceso a historia clinica con usuario, IP y timestamp.

> WhatsApp (Twilio)

Servicio preparado en apps/core/services/whatsapp.py para enviar notificaciones. Simula en modo demo.

> Seed de Demostracion

Comando "python manage.py seed_demo" que carga: 2 medicos, 5 pacientes, 4 turnos, 1 historia clinica.

> Sistema de Login

5 usuarios demo con roles: desarrollador (superuser), admin, recepcion, dr.garcia (medico), dra.martinez (medico).

> Cumplimiento Legal

Ley 25.326 (consentimiento datos personales) y Ley 26.529 (historia clinica con retencion minima 10 anios).

> Diseno Dark Mode

UI oscura (slate-950) con glassmorphism, paleta medica (cyan/blue), Tailwind CSS via CDN.

5. FUNCIONALIDADES PREPARADAS (Pendientes)

> Facturacion AFIP

Modelo FacturaConsulta preparado con tipo comprobante A/B/C y campo CAE. Falta integrar con AFIP.

> Obras Sociales

Modelo ObraSocial con codigo nomenclador. Falta validacion de afiliados y nomenclador.

> Receta Electronica

Campo receta_electronica en HistoriaClinica. Falta generacion y envio digital.

> Google Calendar Sync

Campo google_calendar_id en Medico. Falta integracion con Google Calendar API.

> QR Codes

Libreria qrcode instalada. Pendiente generacion de QR para turnos o pacientes.

> django-otp

Libreria instalada para autenticacion de dos factores. Pendiente integracion.

6. CREDENCIALES DE DEMOSTRACION

Usuario	Contrasena	Rol	Nombre
desarrollador	dev123	Superusuario	Acceso total (licencia/cupos)
admin	admin123	Administrador	Staff de clinica
recepcion	recepcion123	Recepcion	Staff de recepcion
dr.garcia	medico123	Medico	Dr. Alejandro Garcia - Cardiologia
dra.martinez	medico123	Medico	Dra. Sofia Martinez - Pediatria

7. FLUJOS DE USUARIO

7.1 Flujo de Login

1. Usuario accede a /accounts/login/
2. Ingresa usuario y contraseña
3. Django autentica contra User model
4. home_redirect redirige según perfil:
 - Si es medico -> /medico/agenda/
 - Si es recepcion/admin -> /recepcion/

7.2 Flujo de Turno (Recepcion)

1. Recepcion selecciona fecha y medico
2. Ve KPIs: total, en espera, confirmados, atendidos, cancelados
3. Tabla de turnos con acciones:
 - "Anunciar Llegada" -> estado = EN_ESPERA
 - "HC" -> Redirige a historia clinica del paciente
4. Cambio de estado registra en LogAuditoria

7.3 Flujo de Atencion (Medico)

1. Medico ve agenda del día en mobile
2. Cards de turnos con estado actual
3. Botones de acción:
 - "Iniciar" -> estado = EN_ESPERA
 - "Atendido" -> estado = ATENDIDO
 - "Historia Clinica" -> Formulario de evolucion
4. Creación de HC registra en LogAuditoria

7.4 Flujo Turnero TV

1. Smart TV accede a /turnero/ (sin login)
2. Muestra reloj en vivo
3. Pacientes en EN_ESPERA o CONFIRMADO se muestran en lista
4. Pacientes en ATENDIDO se muestran como "Siendo atendidos"
5. Auto-refresh cada 10 segundos

7.5 Flujo Historia Clinica

1. Desde recepcion o agenda del medico
2. Se accede a /pacientes/<id>/historia-clinica/
3. Se registra acceso en LogAuditoria (Ley 26.529)
4. Se muestra timeline de consultas anteriores
5. Formulario para nueva evolucion:
 - Motivo de consulta
 - Diagnostico

- Notas de evolucion
- Tratamiento prescrito

6. Se guarda y se registra creacion en LogAuditoria

8. SEGURIDAD Y CUMPLIMIENTO LEGAL

8.1 Seguridad

- CSRF activado en todas las vistas de formulario

- Login require

8.2 Ley 25.326 - Datos Personales

Consentimiento informado de tratamiento de datos personales. Campo consentimiento_datos en Paciente con timestamp e IP de registro.

8.3 Ley 26.529 - Historia Clinica

Retencion minima de 10 anios de historia clinica. Implementacion via soft-delete (is_deleted + deleted_at). Custom Manager que filtra registros activos por defecto. All_with_deleted para acceso administrativo.

9. DESPLIEGUE

9.1 Desarrollo Local

1. Crear entorno virtual: `python -m venv venv`
2. Activar: `venv\Scripts\activate` (Windows)
3. Instalar dependencias: `pip install -r requirements.txt`
4. Migraciones: `python manage.py migrate`
5. Datos demo: `python manage.py seed_demo`
6. Crear superuser: `python manage.py createsuperuser`
7. Ejecutar: `python manage.py runserver`
8. Acceder a `http://127.0.0.1:8000/`

9.2 Produccion (Heroku/Render)

1. Configurar variable de entorno `DATABASE_URL` (PostgreSQL)
2. Configurar `DJANGO_SECRET_KEY`
3. Configurar `DJANGO_DEBUG=False`
4. El Procfile ejecuta: `gunicorn config.wsgi`
5. WhiteNoise sirve archivos estaticos
6. `python manage.py collectstatic`

10. COMANDOS UTILES

Comando	Descripcion
python manage.py runserver	Iniciar servidor de desarrollo
python manage.py migrate	Aplicar migraciones a la BD
python manage.py seed_demo	Cargar datos de demostracion
python manage.py createsuperuser	Crear superusuario
python manage.py collectstatic	Recopilar archivos estaticos
python manage.py shell	Shell interactiva de Django
python manage.py test	Ejecutar tests
python manage.py dbshell	Consola de la base de datos